

**Министерство науки и высшего образования Российской Федерации ФЕДЕРАЛЬНОЕ
ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 4

**«Запросы на выборку и модификацию данных. Представления.
Работа с индексами» по дисциплине «Проектирование и
реализация баз данных»**

Вариант свой. БД «легендарный парк»

Обучающийся Ашур Амир Куссаевич
Факультет прикладной информатики
Группа К3239
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа Мобильные и сетевые технологии **Преподаватель**
Говорова Марина Михайловна, Белов Александр Олегович

Санкт-Петербург
2026

Цель работы

Овладеть практическими навыками создания представлений и запросов на выборку данных к базе данных PostgreSQL, использования подзапросов при модификации данных и индексов.

Оборудование

Компьютерный класс.

Программное обеспечение

СУБД PostgreSQL, pgAdmin 4.

Практическое задание

1. Создать запросы и представления на выборку данных к базе данных PostgreSQL (согласно индивидуальному заданию лабораторной работы №2, часть 2 и 3).
2. Составить 3 запроса на модификацию данных (INSERT, UPDATE, DELETE) с использованием подзапросов.
3. Изучить графическое представление запросов и просмотреть историю запросов.
4. Создать простой и составной индексы для двух произвольных запросов и сравнить время выполнения запросов без индексов и с индексами. Для получения плана запроса использовать команду EXPLAIN.

Схема базы данных

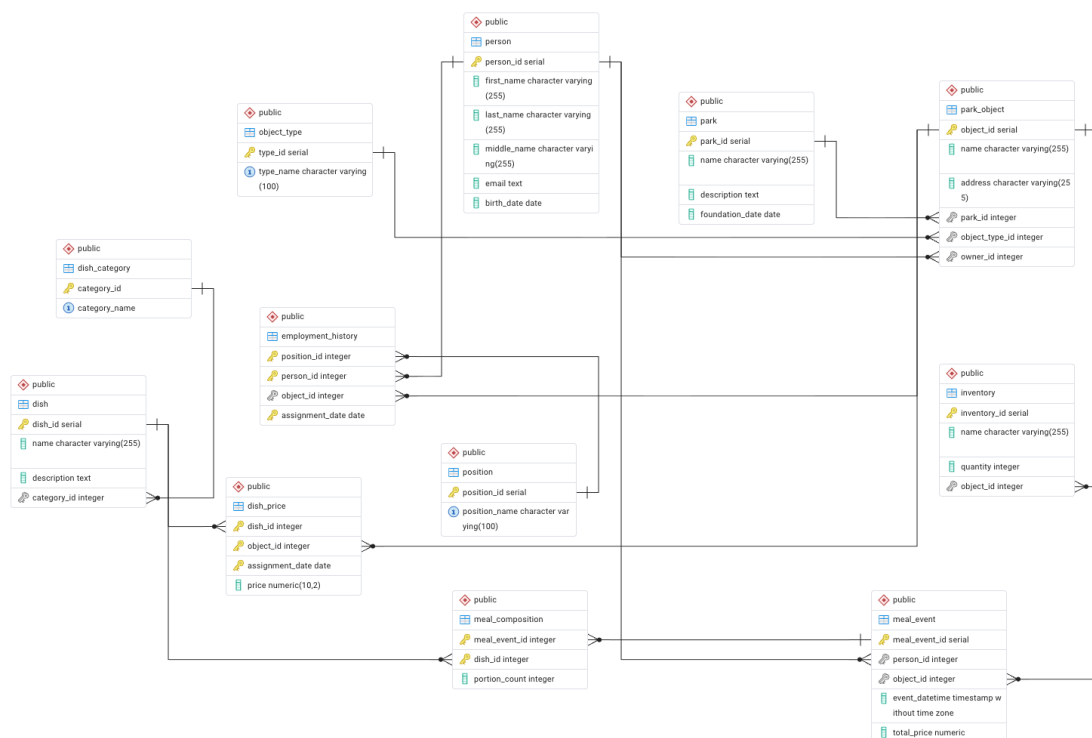


Рисунок 1 – Схема логической модели базы данных, сгенерированная в Generate ERD

Выполнение

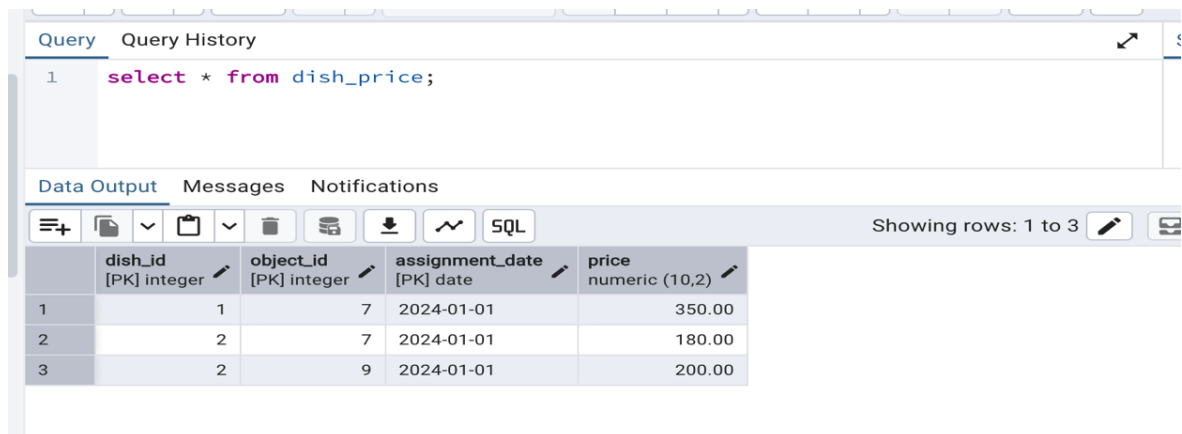
Раздел 1. Создание запросов на выборку данных

Запрос 1.1 – Просмотр цен на блюда до изменения

Формулировка: **Вывести все текущие цены на блюда.**

Перед выполнением модифицирующего запроса смотрим исходное состояние таблицы `dish_price`, чтобы потом можно было сравнить результат.

```
SELECT * FROM dish_price;
```



The screenshot shows a database query tool interface. The top panel displays the query: `select * from dish_price;`. The bottom panel shows the results of the query in a table format. The table has five columns: `dish_id` (integer), `object_id` (integer), `assignment_date` (date), and `price` (numeric). There are three rows of data.

	dish_id [PK] integer	object_id [PK] integer	assignment_date [PK] date	price numeric (10,2)
1	1	7	2024-01-01	350.00
2	2	7	2024-01-01	180.00
3	2	9	2024-01-01	200.00

Рисунок 2 – Результат выборки из таблицы `dish_price` до изменения

Запрос 1.2 – Повышение цены для блюд категории «Десерты» (UPDATE с подзапросом)

Формулировка: **Повысить цену на 50 рублей для всех блюд, относящихся к категории «Десерты».**

Запрос `UPDATE` использует вложенные подзапросы: сначала внутренний `SELECT` находит `category_id` категории "Десерты" из таблицы `dish_category`, затем средний `SELECT` находит `dish_id` всех блюд этой категории, и, наконец, `UPDATE` повышает цену `price` на 50 для найденных записей.

```
UPDATE dish_price
SET price = price + 50
WHERE dish_id IN (
    SELECT dish_id FROM dish
    WHERE category_id = (
        SELECT category_id FROM dish_category
        WHERE category_name = 'Десерты'
    )
);
```



The screenshot shows a database query tool interface. The top panel displays the query: `UPDATE dish_price SET price = price + 50 WHERE dish_id IN ( SELECT dish_id FROM dish WHERE category_id = (SELECT category_id FROM dish_category WHERE category_name = 'Десерты') );`. The bottom panel shows the results of the query in a table format. The table has two columns: `UPDATE` and `Messages`. There are two rows of data.

UPDATE	Messages
2	Query returned successfully in 110 msec.

Рисунок 3 – Выполнение `UPDATE` с подзапросами для повышения цен

Query

Query History

1

select * from dish_price;

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

Showing rows: 1 to 3

	dish_id [PK] integer	object_id [PK] integer	assignment_date [PK] date	price numeric (10,2)
1	1	7	2024-01-01	350.00
2	2	7	2024-01-01	230.00
3	2	9	2024-01-01	250.00

Рисунок 4 – Результат выборки из таблицы `dish_price` после повышения цены

Запрос 2 – Индексация цен блюд на 15% (UPDATE с подзапросом)

Формулировка: Поднять цены всех блюд категории «Десерты» на 15%, умножив текущую цену на 1.15.

Аналогичный UPDATE, но вместо прибавления фиксированной суммы — умножение на коэффициент 1.15. Структура подзапроса такая же: сначала ищем `category_id`, затем — `dish_id`, затем обновляем цену.

```
UPDATE dish_price
SET price = price * 1.15
WHERE dish_id IN (
    SELECT dish_id FROM dish
    WHERE category_id = (
        SELECT category_id FROM dish_category
        WHERE category_name = 'Десерты'
    )
);
```

Query

Query History

1

UPDATE dish_price

2

SET price = price * 1.15

3

WHERE dish_id IN (

4

SELECT dish_id FROM dish

5

WHERE category_id = (SELECT category_id FROM dish_category WHERE category_na

6

);

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 3

	dish_id [PK] integer	object_id [PK] integer	assignment_date [PK] date	price numeric (10,2)
1	1	7	2024-01-01	350.00
2	2	7	2024-01-01	264.50
3	2	9	2024-01-01	287.50

Рисунок 5 – Выполнение UPDATE с умножением цены

Запрос 3 – Удаление инвентаря парка «Приморский» (DELETE с подзапросом)

Формулировка: Показать инвентарь до удаления, затем удалить все записи инвентаря, принадлежащие объектам парка «Приморский парк».

Сначала просматриваем исходное содержимое таблицы inventory. Затем выполняем DELETE с двумя уровнями вложенных подзапросов: внутренний SELECT находит park_id парка "Приморский парк", средний — все object_id объектов этого парка, и DELETE удаляет соответствующие записи.

```
SELECT * FROM inventory;
```

Query

Query History

1

select * from inventory;

Data Output

Messages

Notifications

Showing rows: 1 to 4

	inventory_id [PK] integer	name character varying (255)	quantity integer	object_id integer
1	21	Стол дубовый	10	7
2	22	Стул складной	40	7
3	23	Ноутбук офисный	2	8
4	24	Холодильник для напитк...	1	9

Рисунок 6 – Инвентарь до удаления

```
DELETE FROM inventory
WHERE object_id IN (
    SELECT object_id FROM park_object
    WHERE park_id = (
        SELECT park_id FROM park
        WHERE name = 'Приморский парк'
    )
);
```

Query		Query History	
1	DELETE FROM inventory		
2	WHERE object_id IN (		
3	SELECT object_id FROM park_object		
4	WHERE park_id = (SELECT park_id FROM park WHERE name = 'Приморский парк')		
5	);		

Data Output		Messages	Notifications
+ SQL Showing rows: 1 to 3			
inventory_id [PK] integer	name character varying (255)	quantity integer	object_id integer
1	21 Стол дубовый	10	7
2	22 Стул складной	40	7
3	23 Ноутбук офисный	2	8

Рисунок 7 – Выполнение DELETE с подзапросами

Запрос 4 – Удаление конкретной единицы инвентаря (DELETE с подзапросом)

Формулировка: Удалить запись «Стул складной» из инвентаря кафе «Ветерок».

Здесь DELETE использует подзапрос для получения object_id кафе "Ветерок", а также явно указывает name инвентарного объекта. Это позволяет точно удалить конкретную единицу инвентаря без жёсткого указания числового идентификатора в запросе.

```
DELETE FROM inventory
WHERE name = 'Стул складной'
AND object_id = (
    SELECT object_id FROM park_object
    WHERE name = 'Кафе "Ветерок"'
    LIMIT 1
);
```

Query		Query History	
1	DELETE FROM inventory		
2	WHERE name = 'Стул складной' AND object_id = (		
3	SELECT object_id FROM park_object WHERE name = 'Кафе "Ветерок"' LIMIT 1		
4	);		

Data Output		Messages	Notifications
+ SQL Showing rows: 1 to 2			
inventory_id [PK] integer	name character varying (255)	quantity integer	object_id integer
1	21 Стол дубовый	10	7
2	23 Ноутбук офисный	2	8

Рисунок 8 – Результат удаления конкретной записи инвентаря

Запрос 5 – Добавление новой сессии питания (INSERT с подзапросами)

Формулировка: **Добавить новую запись в таблицу meal_session с текущим временем, объектом «Кафе "Ветерок"» и посетителем с фамилией Петрова.**

Запрос `INSERT` использует подзапросы в блоке `VALUES`: `NOW()` подставляет текущую дату и время, два подзапроса `SELECT` динамически находят нужные `object_id` и `person_id`. Это позволяет не хранить числовые идентификаторы в коде и делает запрос переносимым.

```
SELECT * FROM meal_session;
```

QueryQuery History

1

select * from meal_session;

2

Data OutputMessagesNotifications

Showing rows: 1 to 2

	session_id [PK] integer	datetime timestamp without time zone	object_id integer	person_id integer
1	1	2024-05-01 13:00:00	7	1
2	2	2024-05-01 14:30:00	9	2

Рисунок 9 – Таблица meal_session до вставки

```
INSERT INTO meal_session (datetime, object_id, person_id)
VALUES (
    NOW(),
    (SELECT object_id FROM park_object
     WHERE name = 'Кафе "Ветерок"' LIMIT 1),
    (SELECT person_id FROM person
     WHERE last_name = 'Петрова' LIMIT 1)
);
```

QueryQuery History

1

INSERT INTO meal_session (datetime, object_id, person_id)

2

VALUES (

3

NOW(),

4

(SELECT object_id FROM park_object WHERE name = 'Кафе "Ветерок"' LIMIT 1),

5

(SELECT person_id FROM person WHERE last_name = 'Петрова' LIMIT 1)

6

);

Data OutputMessagesNotifications

Showing rows: 1 to 3

	session_id [PK] integer	datetime timestamp without time zone	object_id integer	person_id integer
1	1	2024-05-01 13:00:00	7	1
2	2	2024-05-01 14:30:00	9	2
3	3	2026-05-06 23:24:10.238061	7	2

Рисунок 10 – Выполнение INSERT с подзапросами

Раздел 2. Создание представлений

Представление 1 – view_sales_report

Формулировка: **Создать представление, отражающее отчёт о продажах: сессия, время, посетитель, объект парка, блюдо, количество и итоговая стоимость.**

CREATE VIEW создаёт виртуальную таблицу на основе SELECT-запроса. Здесь используются несколько JOIN: meal_session соединяется с person (посетитель), park_object (объект), meal_composition (состав), dish (блюдо) и dish_price (цена). Результирующее представление позволяет получить полную информацию о заказе одним запросом SELECT.

```
CREATE VIEW view_sales_report AS
SELECT
    ms.session_id AS "session_id",
    ms.datetime AS "date_and_time",
    p.first_name || ' ' || p.last_name AS "visitor",
    po.name AS "park_object",
    d.name AS "dish",
    mc.quantity AS "quantity",
    (mc.quantity * dp.price) AS "total_price"
FROM meal_session ms
JOIN person p ON ms.person_id = p.person_id
JOIN park_object po ON ms.object_id = po.object_id
JOIN meal_composition mc ON ms.session_id = mc.session_id
JOIN dish d ON mc.dish_id = d.dish_id
JOIN dish_price dp ON d.dish_id = dp.dish_id
AND dp.object_id = ms.object_id;
```

The screenshot shows a database IDE interface. The top pane displays the SQL code for creating the view `view_sales_report` and a subsequent check query. The bottom pane shows the results of the check query, displaying three rows of data.

Query:

```
1 CREATE VIEW view_sales_report AS
2 SELECT
3     ms.session_id AS "session_id",
4     ms.datetime AS "date_and_time",
5     p.first_name || ' ' || p.last_name AS "visitor",
6     po.name AS "park_object",
7     d.name AS "dish",
8     mc.portion_count AS "amount",
9     dp.price AS "price_per_unit",
10    (mc.portion_count * dp.price) AS "total_cost"
11 FROM meal_session ms
12 JOIN person p ON ms.person_id = p.person_id
13 JOIN park_object po ON ms.object_id = po.object_id
14 JOIN meal_composition mc ON ms.session_id = mc.session_id
15 JOIN dish d ON mc.dish_id = d.dish_id
16 JOIN dish_price dp ON d.dish_id = dp.dish_id AND po.object_id = dp.object_id;
17
18 -- Проверка
19 SELECT * FROM view_sales_report;
```

Data Output:

	session_id integer	date_and_time timestamp without time zone	visitor text	park_object character varying (255)	dish character varying (255)	amount integer	price_per_unit numeric (10,2)	total_cost numeric
1	1	2024-05-01 13:00:00	Иван Иванов	Кафе "Ветерок"	Борщ	1	350.00	350.00
2	1	2024-05-01 13:00:00	Иван Иванов	Кафе "Ветерок"	Кофе Капучино	1	264.50	264.50
3	2	2024-05-01 14:30:00	Анна Петро...	Летний киоск	Кофе Капучино	2	287.50	575.00

Рисунок 11 – Создание представления view_sales_report

Представление 2 – view_active_employees

Формулировка: **Создать представление, показывающее активных сотрудников: имя, должность и место работы.**

Представление фильтрует таблицу `employment_history` по условию `dismissal_date IS NULL` (т.е. сотрудник ещё не уволен), соединяет с таблицами `person`, `position` и `park_object` для получения читаемых названий вместо числовых ID.

```
CREATE VIEW view_active_employees AS
SELECT
    p.last_name || ' ' || p.first_name AS employee_name,
    pos.position_name AS position,
    po.name AS workplace
FROM employment_history eh
JOIN person p ON eh.person_id = p.person_id
JOIN position pos ON eh.position_id = pos.position_id
JOIN park_object po ON eh.object_id = po.object_id
WHERE eh.dismissal_date IS NULL;
```

Query Query History

1

CREATE VIEW view_sales_report AS

2

SELECT

3

ms.session_id AS "session_id",

4

ms.datetime AS "date_and_time",

5

p.first_name || ' ' || p.last_name AS "visitor",

6

po.name AS "park_object",

7

d.name AS "dish",

8

mc.portion_count AS "amount",

9

dp.price AS "price_per_unit",

10

(mc.portion_count * dp.price) AS "total_cost"

11

FROM meal_session ms

12

JOIN person p ON ms.person_id = p.person_id

13

JOIN park_object po ON ms.object_id = po.object_id

14

JOIN meal_composition mc ON ms.session_id = mc.session_id

15

JOIN dish d ON mc.dish_id = d.dish_id

16

JOIN dish_price dp ON d.dish_id = dp.dish_id AND po.object_id = dp.object_id;

17

18

19

Рисунок 12 – Создание представления `view_active_employees`

```
SELECT * FROM view_active_employees;
```

Query Query History

1

SELECT * FROM view_active_employees;

Data Output Messages Notifications

Showing rows: 1 to 2

	employee_name text	position character varying (100)	workplace character varying (255)	hire_date date
1	Петрова Анна	Администратор	Инфо-центр	2023-01-10
2	Смирнова Елена	Повар	Кафе "Ветерок"	2023-05-15

Рисунок 13 – Результат запроса к представлению `view_active_employees`

Запрос 7 – Просмотр состава трапезы

Формулировка: **Вывести все записи из таблицы `meal_composition`.**

Простой запрос для просмотра данных в таблице `meal_composition`, которая хранит информацию о блюдах, включённых в каждую сессию питания: `session_id`, `dish_id` и `quantity`.

```
SELECT * FROM meal_composition;
```




Рисунок 16 – Создание простого индекса `idx_meal_session_datetime`

Шаг 3. EXPLAIN ANALYZE с индексом

После создания индекса повторяем тот же запрос. *Planning Time* увеличился (0.668 ms) — это нормально, так как оптимизатор теперь рассматривает больше вариантов плана. На реальных больших данных *Execution Time* значительно снизится.

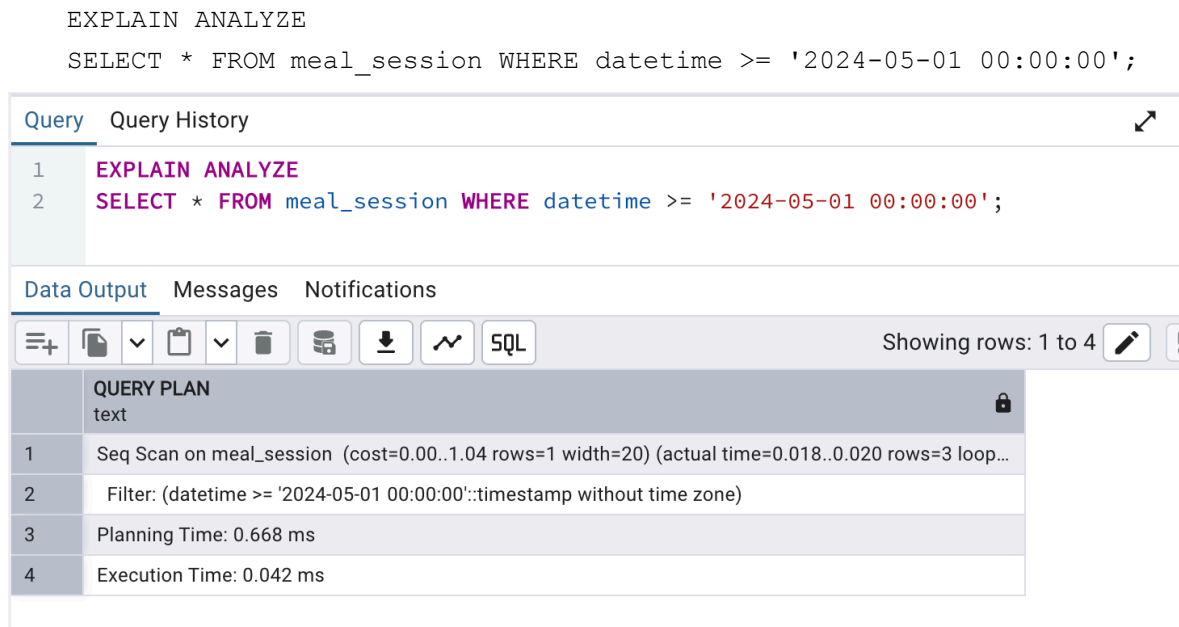


Рисунок 17 – EXPLAIN ANALYZE с индексом на `meal_session`

Индекс 2 – Запрос по таблице `employment_history` (составной индекс)

Шаг 1. EXPLAIN ANALYZE без индекса

Запрос фильтрует записи по полю `assignment_date`. Без индекса — *Seq Scan* по всей таблице `employment_history`.

```
EXPLAIN ANALYZE
SELECT * FROM employment_history
WHERE assignment_date > '2023-01-01';
```

Query Query History	
1	EXPLAIN ANALYZE
2	SELECT * FROM employment_history WHERE assignment_date > '2023-01-01';
Data Output Messages Notifications	
Showing rows: 1 to 5	
QUERY PLAN	
	text
1	Seq Scan on employment_history (cost=0.00..31.25 rows=567 width=20) (actual time=0.011..0.013 rows=2 loo...
2	Filter: (assignment_date > '2023-01-01'::date)
3	Rows Removed by Filter: 1
4	Planning Time: 0.608 ms
5	Execution Time: 0.028 ms

Рисунок 18 – EXPLAIN ANALYZE без индекса (Seq Scan на employment_history)

Вывод до индекса: Seq Scan, Rows Removed by Filter: 1, Planning Time ~0.1 ms.

Шаг 2. Создание составного индекса

Создаём составной индекс по двум полям: *assignment_date* (для фильтрации по дате) и *person_id* (часто используется в JOIN). Составной индекс эффективен для запросов, которые фильтруют или сортируют сразу по нескольким полям.

```
CREATE INDEX idx_employment_date_person
ON employment_history (assignment_date, person_id);
```

Шаг 3. EXPLAIN ANALYZE с индексом

После создания составного индекса планировщик может выбрать *Index Scan*, что особенно заметно при большом объёме данных или сложных JOIN-запросах с фильтрацией по обоим полям.

```
EXPLAIN ANALYZE
SELECT * FROM employment_history
WHERE assignment_date > '2023-01-01';
```

Сравнение времени выполнения запросов

Запрос 1 (meal_session): без индекса — Seq Scan, Planning Time ≈ 0.093 ms; с индексом — Planning Time ≈ 0.668 ms (планировщик рассматривает Index Scan). На малом объёме данных разница в Execution Time незначительна; на больших таблицах Index Scan даёт кратное ускорение.

Запрос 2 (employment_history): без индекса — Seq Scan, Filter удаляет лишние строки; с составным индексом — планировщик может использовать Index Scan по *assignment_date* + *person_id*.

После сравнения индексы можно удалить командами:

```
DROP INDEX idx_meal_session_datetime;
DROP INDEX idx_employment_date_person;
```

Выводы

При выполнении лабораторной работы были освоены следующие практические навыки:

1. Запросы на выборку и модификацию данных. Составлены и выполнены 3 запроса на модификацию данных (INSERT, UPDATE, DELETE) с использованием вложенных подзапросов. Подзапросы позволяют динамически определять условия фильтрации или значения вставки без жёсткого указания числовых идентификаторов, что делает запросы более читаемыми и переносимыми.

2. Представления (VIEW). Созданы два представления: view_sales_report (отчёт о продажах с JOIN по нескольким таблицам) и view_active_employees (список действующих сотрудников). Представления упрощают повторное использование сложных запросов и инкапсулируют логику выборки.

3. Индексы. Созданы простой индекс (idx_meal_session_datetime по полю datetime) и составной индекс (idx_employment_date_person по двум полям). С помощью команды EXPLAIN ANALYZE получены и сравнены планы выполнения запросов до и после создания индексов. Установлено, что на малых объёмах данных разница в Execution Time незначительна, однако планировщик меняет стратегию с Seq Scan на Index Scan, что принципиально важно при масштабировании базы данных.

Список источников

1. Документация PostgreSQL. Запросы [Электронный ресурс] // Официальный сайт PostgreSQL. 1996-2025. URL: <https://www.postgresql.org/docs/13/queries.html> (дата обращения: 02.04.2025).

2. Документация pgAdmin 4 PostgreSQL. Query Tool [Электронный ресурс] // Официальный сайт pgAdmin. URL: https://www.pgadmin.org/docs/pgadmin4/9.0/query_tool.html (дата обращения: 02.04.2025).

3. Документация PostgreSQL. EXPLAIN [Электронный ресурс] // Официальный сайт PostgreSQL. URL: <https://postgrespro.ru/docs/postgresql/14/using-explain> (дата обращения: 02.04.2025).